Project Report: BloodLife Management System

Technical & Architectural Specification | August 2026

Executive Summary

BloodLife is a web application for connecting voluntary blood donors with people who need blood transfusions. Users can search for donors by blood group and location, submit registration forms to become donors, and manage donor records through an administrative dashboard.

The application supports multiple database backends: Supabase (cloud PostgreSQL), direct PostgreSQL, and local MySQL via XAMPP. It falls back automatically based on available environment variables.

1. Objectives and Scope

Objectives

- **Fast Search:** Help users search for and contact blood donors quickly.
- **Simple Registration:** Allow voluntary donors to submit contact information, blood type, age, location, and availability.
- **Admin Governance:** Provide administrators with tools to view statistics, add donors, and delete records.
- **Flexible Environment Support:** Support local offline running and cloud deployment with automatic database fallback.

Scope of Work

- **Public Features:** Donor search, blood group filters for all 8 ABO/Rh types (A+, A-, B+, B-, AB+, AB-, O+, O-), and a donor registration form with client-side validation.
- **Admin Features:** Password-protected login, statistics overview (total and available donors), donor table with modal creation form, and record deletion.
- **API:** JSON REST endpoints for data fetching and state updates.

2. Architecture and Technology Stack

The application uses a decoupled model where the Flask backend exposes REST API endpoints consumed by vanilla JavaScript on the frontend. Database connections adjust dynamically based on environment configuration.

Technology Stack Table

Layer	Technology	Function
Backend	Python 3 / Flask	HTTP routing, session management, REST API controllers
Database Drivers	Supabase SDK / psycopg2 / PyMySQL	Database connections with automatic fallback
Frontend	HTML5, CSS3, JS (ES6)	Responsive UI, AJAX calls, flexbox and grid layouts
Templating	Jinja2	Layout inheritance through base.html
Deployment	Vercel / XAMPP	Serverless deployment on Vercel or local execution with XAMPP

3. Database Design

donors Table Schema

Field Name	Type	Constraints	Description
id	SERIAL / INT	PRIMARY KEY	Unique donor ID
name	VARCHAR(100)	NOT NULL	Donor full name
blood_group	VARCHAR(5)	NOT NULL	ABO and Rh factor (A+, B+, etc.)
phone	VARCHAR(20)	NOT NULL	Contact phone number
email	VARCHAR(100)	NULLABLE	Email address
location	VARCHAR(100)	NOT NULL	City or area name
address	TEXT	NULLABLE	Street address
age	INT	NULLABLE	Donor age (18 to 65)
gender	VARCHAR(10)	NULLABLE	Gender selection
is_available	BOOLEAN	DEFAULT TRUE	Current availability
created_at	TIMESTAMP	CURRENT_TIMESTAMP	Record creation time

4. Key Application Features

Donor Search (/donors)

- **Real-Time Search:** Filter donors by name, area, or phone number in real time.
- **Blood Group Quick-Filters:** Filter by blood group using pill buttons for each group.
- **Direct Calling:** Clickable call buttons (tel: format) trigger phone dialers.

Donor Registration (/register)

- **Field Validation:** Form validation for required fields: name, blood group, phone, and location.
- **Feedback Messages:** Immediate success and error messages without full page reloads.

Admin Dashboard (/admin/dashboard)

- **Live Statistics:** Real-time counts of total registered donors and available donors.
- **Modal Donor Entry:** Modal dialog for adding new donor entries.
- **Record Deletion:** One-click deletion for donor records.

5. API Reference

- **GET /api/stats:** Returns system counts and donor distribution data.
- **GET /api/donors:** Returns donor list with optional search, blood, and location filters.
- **POST /api/donors:** Adds a new donor record.
- **DELETE /api/donors/<id>:** Deletes a donor record (requires admin session).
- **POST /api/login & /api/logout:** Manages admin sessions.

6. Setup and Deployment

Local Setup (XAMPP / MySQL)

1. Install requirements: pip install -r requirements.txt
2. Start MySQL in XAMPP and import database/init.sql through phpMyAdmin.
3. Run python app.py or run.bat.

Cloud Deployment (Supabase & Vercel)

1. Run database/supabase_init.sql in the Supabase SQL Editor.
2. Set SUPABASE_URL and SUPABASE_KEY in Vercel environment variables.

7. Future Work

- **SMS Verification:** SMS verification for donor phone numbers using Twilio.
- **Interactive Mapping:** Interactive map view for location-based donor search.
- **Donor Self-Service:** Donor account portal for updating availability status.